Gestión de proyectos de inteligencia artificial

Semana 8, Módulo 2, Fase 2

Tomás Olivares - 08/11/26

Fase 2 — Priorización predictiva de mantenimiento | Grupo Bimbo

Prototipo académico de una capa de IA para priorizar señales de riesgo de falla de activos industriales. Continúa el business case de Fase 1: combina condición del activo, historial de mantenimiento y criticidad. La IA se mantiene sólo como "recomendador"; el técnico conserva la decisión final. La actividad "Fase 1" definió el business case, pero no entregó un modelo entrenado, dataset ni API. Este repositorio implementa un prototipo demostrativo nuevo con datos sintéticos reproducibles. No utiliza datos internos de Grupo Bimbo.

Repositorio en github: https://github.com/tomasolvs/master-ai-gestion-proyectos-ai-fase-2

```
!git clone https://github.com/tomasolvs/master-ai-gestion-proyectos-ai-fase-2.git

%cd /content/master-ai-gestion-proyectos-ai-fase-2
```

```
Cloning into 'master-ai-gestion-proyectos-ai-fase-2'...
remote: Enumerating objects: 32, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 32 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (32/32), 2.40 MiB | 2.90 MiB/s, done.
Resolving deltas: 100% (5/5), done.
/content/master-ai-gestion-proyectos-ai-fase-2
```

```
%pip install -q -r requirements.txt
```

```
import fastapi
import joblib
import sklearn
import pandas
import numpy

print("FastAPI:", fastapi.__version__)
print("Joblib:", joblib.__version__)
print("Scikit-learn:", sklearn.__version__)
print("Dependencias cargadas correctamente.")
```

```
FastAPI: 0.115.12
Joblib: 1.4.2
Scikit-learn: 1.6.1
Dependencias cargadas correctamente.
```

```
!python scripts_train.py
```

```
Modelo entrenado y guardado.
ROC-AUC: 0.8091
Matriz de confusión: [[452, 43], [54, 51]]
```

```
import subprocess
import sys
import time
import requests

api_log = open("/content/uvicorn.log", "w")

api_process = subprocess.Popen(
    [
        sys.executable,
        "-m", "uvicorn",
        "app.main:app",
        "--host", "0.0.0.0",
        "--port", "8000"
    ],
    cwd=os.getcwd(),
    stdout=api_log,
    stderr=subprocess.STDOUT
)

time.sleep(5)

print("Proceso activo:", api_process.poll() is None)

with open("/content/uvicorn.log") as f:
    print(f.read())
```

```
Proceso activo: True
INFO:     Started server process [1877]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

```
response = requests.get(
    "http://127.0.0.1:8000/health"
)

print("HTTP Status:", response.status_code)
print("Respuesta:", response.text)

if response.status_code == 200:
    print("API funcionando correctamente.")
else:
    print("La API todavía presenta un error.")
```

```
HTTP Status: 200
Respuesta: {"status":"ok","model_loaded":true}
API funcionando correctamente.
```

```
payload = {
    "asset_id": "MOTOR-001",
    "temperature_c": 92,
    "vibration_mm_s": 8.2,
    "pressure_bar": 6.5,
    "energy_kw": 48,
    "operating_hours": 18000,
    "work_orders_90d": 5,
    "days_since_maintenance": 220,
    "asset_criticality": 5
}

response = requests.post(
    "http://127.0.0.1:8000/predict",
    json=payload
)

print("HTTP:", response.status_code)
print(response.text)
```

```
HTTP: 200
{"asset_id":"MOTOR-001","failure_probability":0.5659,"risk_level":"MEDIO","priority":"P2 - programar revisión","contributing_signals":["Temperatura elevada (≥90 °C)","Vibración elevada (≥7.1 mm/s)","Órdenes de trabajo frecu
```

```
# Verificar que el frontend está disponible
frontend = requests.get("http://127.0.0.1:8000/")

print("Frontend:", frontend.status_code)
print("HTML recibido:", "text/html" in frontend.h
```

```
assert frontend.status_code == 200

# Verificar documentación automática
docs = requests.get("http://127.0.0.1:8000/docs")

print("Swagger:", docs.status_code)

assert docs.status_code == 200

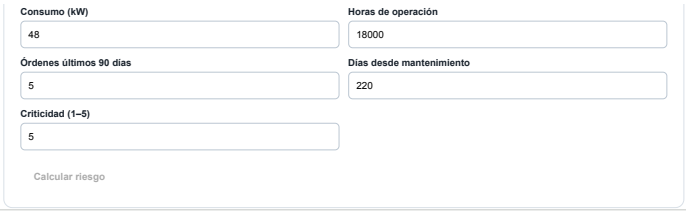
print("Frontend y documentación API disponibles."

from google.colab import output
from IPython.display import IFrame, display

frontend_url = output.eval_js(
    "google.colab.kernel.proxyPort(8000)"
)

display(
    IFrame(
        frontend_url,
        width="100%",
        height=700
    )
)
```

```
Frontend: 200
HTML recibido: True
Swagger: 200
Frontend y documentación API disponibles.
```



Comienza a programar o generar con IA.

```
# Caso extremo válido
extreme = payload.copy()

extreme.update({
    "temperature_c": 150,
    "vibration_mm_s": 25,
    "work_orders_90d": 30,
    "days_since_maintenance": 1000,
    "asset_criticality": 5
})

r = requests.post(
    "http://127.0.0.1:8000/predict",
    json=extreme
)

print("Caso extremo:", r.status_code)
print(r.json())

assert r.status_code == 200

# Caso inválido: criticidad superior al máximo permitido
invalid = payload.copy()
invalid["asset_criticality"] = 8

r = requests.post(
    "http://127.0.0.1:8000/predict",
    json=invalid
)

print("Caso inválido:", r.status_code)

assert r.status_code == 422

print("Ambos escenarios validados correctamente.")
```

```
Caso extremo: 200
{'asset_id': 'MOTOR-001', 'failure_probability': 0.8687, 'risk_level': 'ALTO', 'priority': 'P1 – revisión prioritaria', 'contributing_signals': ['Temperatura elevada (≥90 °C)', 'Vibración elevada (≥7.1 mm/s)', 'Órdenes de t
Caso inválido: 422
Ambos escenarios validados correctamente.
```

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "-m", "pytest", "-v"],
    capture_output=True,
    text=True
)

print(result.stdout)
print(result.stderr)

assert result.returncode == 0, "Las pruebas autom
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-8.3.4, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /content/master-ai-gestion-proyectos-ai-fase-2
plugins: langsmith-0.14.17, typeguard-4.6.0, anyio-4.16.0, platformdirs-4.14.3
collecting ... collected 6 items

tests/test_api.py::test_health PASSED                                    [ 16%]
tests/test_api.py::test_prediction_contract PASSED                       [ 33%]
tests/test_api.py::test_extreme_but_valid_reading PASSED                 [ 50%]
tests/test_api.py::test_invalid_criticality_rejected PASSED              [ 66%]
tests/test_api.py::test_negative_vibration_rejected PASSED               [ 83%]
tests/test_api.py::test_unknown_field_rejected PASSED                    [100%]
```

```
============================= 6 passed in 2.56s ==============================
```